Isaac Seip
CMPSC 448 Midterm Project

1. Introduction

Different LLMs may produce responses with distinct stylistic and linguistic differences, even when answering the same prompt. This project investigates whether machine learning can be used to identify which LLM generated a particular response. I implemented both CNN and LSTM architectures to investigate these research questions:

RQ1: Can we identify which LLM generated a response using a CNN or RNN?

RQ2: Does the user's prompt help identify the LLM?

I compare the accuracy of both models using input only, output only, and both input and output.

2. Dataset and Preprocessing

I chose to use a publicly available dataset for this project. I used this dataset from Hugging Face: geometriqs/global80_prompt-response-pairs. I chose this dataset because it contained the original prompts, all of the generated responses, and which model the response originated from. It also has models from three different LLM families, specifically Open AI's GPT-5, Google's Gemini 2.5 Flash, and Perplexity's Sonar. The dataset also identifies which responses used web-search grounding. I filtered out all of the responses using the search grounding so the models could be evaluated under the same conditions. I then removed any missing or empty values as well as any duplicate examples. After cleaning and filtering, I ended with 677 examples with 231 belonging to Sonar, 228 belonging to GPT-5, and 218 belonging to Gemini.

I did an 80/20 split on the data, which resulted in 538 training examples and 139 testing examples. I split the data by unique prompts because the models answered the same prompts. Using unique prompts prevents responses with the same prompt being placed in both training and testing.

I converted the text to lowercase and split it into tokens using spaces. Then I created a vocabulary of up to 10,000 different tokens using the responses in the training set. These tokens then get encoded into numbers. I padded or shortened the encoded sequences to a max length of 400 tokens, so the inputs would be consistent.

3. Model Implementation

CNN : The CNN begins with an embedding layer, which converts numerical tokens into 64-dimensional vectors, allowing the model to learn the word representations. The convolution layer uses 64 filters with a kernel size of 3 to analyze groups of three tokens that neighbor each other. ReLU allows the network to learn more complicated patterns instead of being limited to simple relationships. Max pooling selects the strongest features across the text. The linear layer takes 64 input features to produce scores for the three LLMs.

LSTM: Same as CNN, the LSTM begins with an embedding layer that converts the tokens into 64-dimensional vectors. The LSTM layer uses 64 hidden units and sequentially processes the tokens. Its internal memory helps it store information from earlier parts of the sequence. Mean pooling averages the LSTM outputs across the sequence to produce one representation of the input text. The linear layer converts that representation into three classification scores.

The CNN focuses on patterns between neighboring words, while LSTM can learn relationships between tokens across a sequence. Using both, we can see which is more useful for identifying which LLM generated a specific response.

Both models were implemented using PyTorch with cross-entropy loss used to measure error. I set a manual seed of 42, so the experiment results were easier to reproduce. I used Adam optimizer with the default learning rate of 0.001. Each model went through 10 full passes of training before evaluating the accuracy.

4. Results and Analysis

Input Type	**CNN Accuracy**	**LSTM Accuracy**
Prompt Only	33.81%	33.81%
Response Only	65.47%	61.15%
Prompt and Response	61.87%	65.47%

Both models achieved 33.81% accuracy with only the prompt. This is very close to the accuracy you'd expect from randomly guessing. This makes sense because the models received the same prompts, so the prompt alone gives little information about the LLM.

With the results only using the response, the models performed significantly better with CNN having an accuracy of 65.47% and LSTM having an accuracy of 61.15%. The CNN performed slightly better than the LSTM, although this doesn't mean it will always be better at identifying the specific LLM.
When both the prompt and response were used, CNN performed slightly worse, while LSTM performed better. I would have expected the prompt to improve classification, but the CNN results show that the additional information doesn't necessarily improve performance. This could be because my max length limit causes parts of some responses to be cut off. Both CNN and LSTM achieved a maximum accuracy of 65.47%, but with different input conditions. Neither model consistently outperformed the other across the input combinations.

5. Limitations and Lessons Learned

There were a few different limitations in my project. After filtering and cleaning, I was only left with 677 examples. A larger dataset with more examples could've improved the training data. The vocabulary I used was built from the responses only, so words that appeared in the prompt but not the response may become unknown tokens. Another possible limitation was that my max length limit was 400. This means that longer responses could be cut off, especially when both the prompt and response are used for input.

I learned that more information in the input isn't guaranteed to improve the accuracy. I saw this in my CNN results using both the prompt and response. I was surprised because I expected the combined input to perform better. This project helped show me how important preparing the dataset is before building the models. The text needs to be tokenized and converted into numbers before the models can process it. I also learned that reproducibility is very important. My resulting accuracies varied, so I added a seed to make sure my results were easy to reproduce. If I were to continue this project, I would experiment with a larger dataset and different max sequence lengths to see whether the accuracy improves.